

Celebal Summer Internship 2026

Week 2 Task: E-Commerce Sales Database

Database Setup:

i. Create Database-

```
CREATE DATABASE shopease_db;
```

```
USE shopease_db;
```

ii. Create Tables-

Copy and run the 4 CREATE TABLE statements from the assignment **in this order**:

1. customers
2. products
3. orders
4. order_items

iii. Insert Sample data & Verify data loaded

ASSIGNMENT QUES

SECTION-A

Q1. Write a query to display all columns and rows from the customer's table.

```
SELECT * FROM customers;
```

Result Grid								
Filter Rows:								
	customer_id	first_name	last_name	email	city	state	join_date	is_premium
▶	101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
	102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0
	103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1
	104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0
	105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1
	106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0
	107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
	108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	0
✱	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Q2. Retrieve only the first_name, last_name, and city of all customers.

```
SELECT first_name,
       last_name,
       city
FROM customers;
```

Result Grid			
Filter Rows:			
	first_name	last_name	city
▶	Aarav	Sharma	Mumbai
	Priya	Patel	Ahmedabad
	Rohan	Gupta	Delhi
	Sneha	Reddy	Hyderabad
	Vikram	Singh	Jaipur
	Ananya	Iyer	Chennai
	Karan	Mehta	Pune
	Divya	Nair	Kochi

Q3. List all unique categories available in the products table.

```
SELECT DISTINCT category
FROM products;
```

Result Grid	
Filter Rows:	
	category
▶	Clothing
	Electronics
	Home

Q4. Identify the Primary Key of each table in the schema. Explain why a Primary Key must be unique and NOT NULL.

- Check Primary key for Customers

```
SHOW KEYS
FROM customers
```

WHERE Key_name='PRIMARY';

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment	Index_comment	Visible	Expression
customers	0	PRIMARY	1	customer_id	A	8	NULL	NULL		BTREE			YES	NULL

- ii. Check Primary key for Products

SHOW KEYS

FROM products

WHERE Key_name='PRIMARY';

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment	Index_comment	Visible	Expression
products	0	PRIMARY	1	product_id	A	8	NULL	NULL		BTREE			YES	NULL

- iii. Check Primary key for Orders

SHOW KEYS

FROM orders

WHERE Key_name='PRIMARY';

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment	Index_comment	Visible	Expression
orders	0	PRIMARY	1	order_id	A	10	NULL	NULL		BTREE			YES	NULL

- iv. Check Primary Key of Order_Items

SHOW KEYS

FROM order_items

WHERE Key_name='PRIMARY';

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment	Index_comment	Visible	Expression
order_items	0	PRIMARY	1	item_id	A	15	NULL	NULL		BTREE			YES	NULL

Q5. What constraints are applied to the email column in the customers table? What would happen if you tried to insert a duplicate email?

Column Definition

email VARCHAR(100) UNIQUE NOT NULL

Constraints

1. UNIQUE

- No duplicate email addresses allowed.

2. NOT NULL

- Email cannot be empty.

Example of Duplicate Email

```
INSERT INTO customers VALUES (109, 'Rahul', 'Kumar', 'aarav.s@email.com', 'Mumbai', 'Maharashtra', '2024-09-01', FALSE );
```

Result

MySQL Error:

Duplicate entry 'aarav.s@email.com' for key 'customers.email'

Explanation

The UNIQUE constraint prevents duplicate email addresses.

Q6. Try inserting a product with unit_price = -50. What happens and which constraint prevents it? Write both the INSERT statement and explain the error.

Query:

```
INSERT INTO products VALUES(209, 'Bad Product', 'Electronics', 'ABC', -50, 100 );
```

Constraint Responsible

```
CHECK (unit_price > 0)
```

Result

MySQL will reject the insertion and return an error similar to:

Check constraint 'products_chk_1' is violated

Explanation

The table requires:

```
unit_price > 0
```

Since:

```
-50 > 0 → FALSE
```

the record violates the CHECK constraint and is not inserted.

SECTION B- Filtering & Optimization (WHERE, Indexes)

Q7. Retrieve all orders with status = 'Delivered'.

```
SELECT * FROM orders WHERE status='Delivered';
```

Result Grid						Filter Rows:	Edit:	Export/Import:	Wrap Cell Content:
	order_id	customer_id	order_date	status	total_amount				
▶	1001	101	2024-08-01	Delivered	4498.00				
	1002	102	2024-08-03	Delivered	799.00				
	1004	101	2024-08-10	Delivered	3499.00				
	1006	105	2024-08-15	Delivered	5898.00				
	1008	103	2024-08-20	Delivered	899.00				
	1010	108	2024-08-28	Delivered	1598.00				
*	NULL	NULL	NULL	NULL	NULL				

Q8. Find all products in the 'Electronics' category with a unit_price greater than ₹2000.

SELECT * FROM products WHERE category='Electronics' AND unit_price > 2000;

Result Grid						Filter Rows:	Edit:	Export/Import:	Wrap Cell Content:
	product_id	product_name	category	brand	unit_price	stock_qty			
▶	203	Smart Watch	Electronics	Noise	2999.00	150			
	205	Bluetooth Speaker	Electronics	JBL	3499.00	200			
*	NULL	NULL	NULL	NULL	NULL	NULL			

Q9. List all customers who joined in the year 2024 and belong to the state 'Maharashtra'.

Query

SELECT * FROM customers
WHERE state = 'Maharashtra'
AND join_date BETWEEN '2024-01-01' AND '2024-12-31';

Result Grid								Filter Rows:	Edit:	Export/Import:	Wrap Cell Content:
	customer_id	first_name	last_name	email	city	state	join_date	is_premium			
▶	101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1			
	107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1			
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL			

Q10. Find all orders placed between '2024-08-10' and '2024-08-25' (inclusive) that are NOT cancelled.

Query

SELECT *
FROM orders
WHERE order_date BETWEEN '2024-08-10' AND '2024-08-25'
AND status <> 'Cancelled';

Result Grid	Filter Rows:	Edit:	Export/Import:	Wrap Cell Content:
order_id	customer_id	order_date	status	total_amount
1004	101	2024-08-10	Delivered	3499.00
1006	105	2024-08-15	Delivered	5898.00
1007	106	2024-08-18	Pending	1299.00
1008	103	2024-08-20	Delivered	899.00
1009	107	2024-08-25	Shipped	6098.00
NULL	NULL	NULL	NULL	NULL

Q11. Explain what the index `idx_orders_date` does. How would it improve the performance of a query that filters orders by `order_date`? Write a sample query that would benefit from this index.

Definition

```
CREATE INDEX idx_orders_date
ON orders(order_date);
```

Purpose

An index works like the index of a book.

Without an index:

- The database scans every row.
- This is called a **Full Table Scan**.

With an index:

- The database can directly locate matching dates.
- Query execution becomes faster, especially for large tables.

Example Query Benefiting from the Index

```
SELECT *
FROM orders
WHERE order_date BETWEEN '2024-08-01'
AND '2024-08-31';
```

Result Grid					
	order_id	customer_id	order_date	status	total_amount
▶	1001	101	2024-08-01	Delivered	4498.00
	1002	102	2024-08-03	Delivered	799.00
	1003	103	2024-08-05	Shipped	7498.00
	1004	101	2024-08-10	Delivered	3499.00
	1005	104	2024-08-12	Cancelled	2999.00
	1006	105	2024-08-15	Delivered	5898.00
	1007	106	2024-08-18	Pending	1299.00
	1008	103	2024-08-20	Delivered	899.00
	1009	107	2024-08-25	Shipped	6098.00
	1010	108	2024-08-28	Delivered	1598.00
*	NULL	NULL	NULL	NULL	NULL

Performance Benefit

Instead of checking every order record, MySQL can use the index to quickly find rows within the requested date range.

Q12. If you run: `SELECT * FROM customers WHERE YEAR(join_date) = 2024;` — would the index on `join_date` be used? Explain why or why not, and rewrite the query to be index-friendly (SARGable).

Query

`SELECT * FROM customers WHERE YEAR(join_date) = 2024;`

Result Grid					
	order_id	customer_id	order_date	status	total_amount
▶	1001	101	2024-08-01	Delivered	4498.00
	1002	102	2024-08-03	Delivered	799.00
	1003	103	2024-08-05	Shipped	7498.00
	1004	101	2024-08-10	Delivered	3499.00
	1005	104	2024-08-12	Cancelled	2999.00
	1006	105	2024-08-15	Delivered	5898.00
	1007	106	2024-08-18	Pending	1299.00
	1008	103	2024-08-20	Delivered	899.00
	1009	107	2024-08-25	Shipped	6098.00
	1010	108	2024-08-28	Delivered	1598.00
*	NULL	NULL	NULL	NULL	NULL

Answer

No, the index would generally not be used efficiently.

Why?

The function:

`YEAR(join_date)`

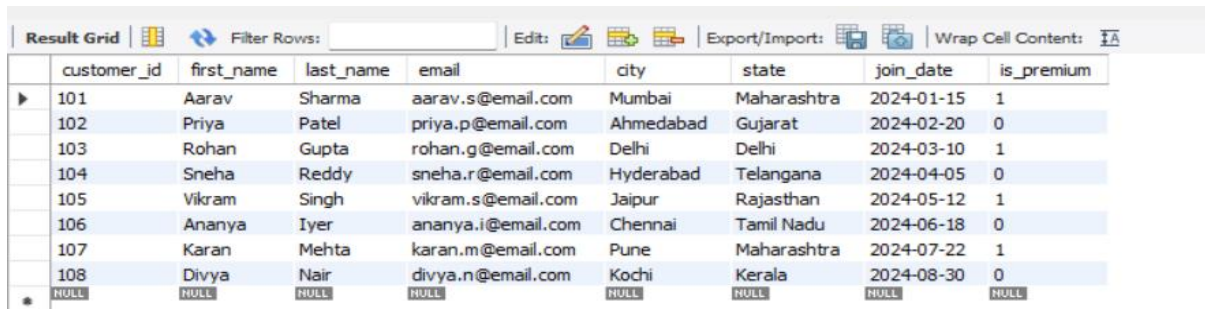
is applied to every row before comparison.

Because the column is wrapped inside a function, MySQL cannot efficiently use the index and may perform a full table scan.

This is called a **Non-SARGable Query**.

Index-Friendly (SARGable) Version

```
SELECT *  
FROM customers  
WHERE join_date >= '2024-01-01'  
AND join_date < '2025-01-01';
```



The screenshot shows a database query result grid with columns: customer_id, first_name, last_name, email, city, state, join_date, and is_premium. The data is as follows:

customer_id	first_name	last_name	email	city	state	join_date	is_premium
101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0
103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1
104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0
105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1
106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0
107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	0
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Why is this Better?

- No function is applied to join_date.
- The database can directly use the index on the date column.
- Faster performance on large datasets.

SECTION C- Aggregation (GROUP BY, SUM, COUNT, AVG, MIN, MAX)

Q13. Count the total number of orders in the orders table.

```
SELECT COUNT(*) AS total_orders  
FROM orders;
```



The screenshot shows a database query result grid with a single column labeled total_orders and a single row with the value 10.

total_orders
10

Q14. Find the total revenue (SUM of total_amount) from all 'Delivered' orders.


```
SELECT SUM(total_amount) AS total_revenue
FROM orders
WHERE status='Delivered';
```

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	total_revenue				
▶	17191.00				

Q15. Calculate the average unit_price of products in each category.

```
SELECT category,
       AVG(unit_price) AS avg_price
FROM products
GROUP BY category;
```

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	category	avg_price			
▶	Clothing	2699.000000			
	Electronics	2224.000000			
	Home	949.000000			

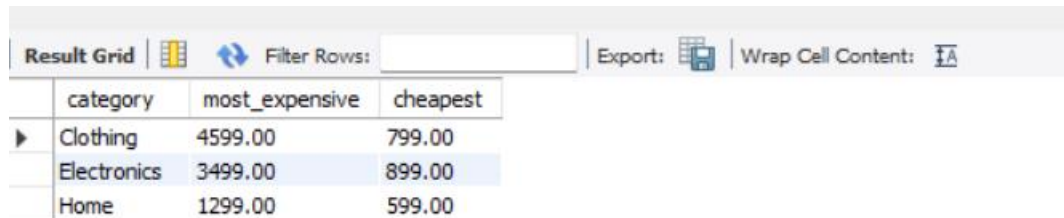
Q16. For each order status, find the count of orders and the total revenue. Sort the result by total revenue in descending order.

```
SELECT status,
       COUNT(*) AS order_count,
       SUM(total_amount) AS total_revenue
FROM orders
GROUP BY status
ORDER BY total_revenue DESC;
```

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	status	order_count	total_revenue		
▶	Delivered	6	17191.00		
	Shipped	2	13596.00		
	Cancelled	1	2999.00		
	Pending	1	1299.00		

Q17. Find the most expensive (MAX) and cheapest (MIN) product in each category.

```
SELECT category,  
       MAX(unit_price) AS most_expensive,  
       MIN(unit_price) AS cheapest  
FROM products  
GROUP BY category;
```



The screenshot shows a database query result grid with the following data:

	category	most_expensive	cheapest
▶	Clothing	4599.00	799.00
	Electronics	3499.00	899.00
	Home	1299.00	599.00

Q18. List all product categories where the average unit_price is greater than ₹2000. (Hint: Use HAVING clause)

```
SELECT category,  
       AVG(unit_price) AS avg_price  
FROM products  
GROUP BY category  
HAVING AVG(unit_price) > 2000;
```



The screenshot shows a database query result grid with the following data:

	category	avg_unit_price
▶	Clothing	2699.000000
	Electronics	2224.000000

SECTION D - Joins & Relationships

Q19. Write an INNER JOIN query to display each order along with the customer's first_name and last_name.

Show: order_id, order_date, first_name, last_name, total_amount.

```
SELECT  
       o.order_id,  
       o.order_date,
```

```
c.first_name,  
c.last_name,  
o.total_amount
```

```
FROM orders o
```

```
INNER JOIN customers c
```

```
ON o.customer_id = c.customer_id;
```

Result Grid					
		Filter Rows:			
		Export:	Wrap Cell Content:		
	order_id	order_date	first_name	last_name	total_amount
▶	1001	2024-08-01	Aarav	Sharma	4498.00
	1004	2024-08-10	Aarav	Sharma	3499.00
	1002	2024-08-03	Priya	Patel	799.00
	1003	2024-08-05	Rohan	Gupta	7498.00
	1008	2024-08-20	Rohan	Gupta	899.00
	1005	2024-08-12	Sneha	Reddy	2999.00
	1006	2024-08-15	Vikram	Singh	5898.00
	1007	2024-08-18	Ananya	Iyer	1299.00
	1009	2024-08-25	Karan	Mehta	6098.00
	1010	2024-08-28	Divya	Nair	1598.00

Q20. Using a LEFT JOIN, list ALL customers and their orders (if any). Customers with no orders should still appear with NULL values for order columns.

```
SELECT
```

```
c.customer_id,  
c.first_name,  
c.last_name,  
o.order_id,  
o.order_date,  
o.status
```

```
FROM customers c
```

```
LEFT JOIN orders o
```

```
ON c.customer_id = o.customer_id;
```

Result Grid						
Filter Rows:						
Export: Export						
Wrap Cell Content: Wrap						
	customer_id	first_name	last_name	order_id	order_date	status
▶	101	Aarav	Sharma	1001	2024-08-01	Delivered
	101	Aarav	Sharma	1004	2024-08-10	Delivered
	102	Priya	Patel	1002	2024-08-03	Delivered
	103	Rohan	Gupta	1003	2024-08-05	Shipped
	103	Rohan	Gupta	1008	2024-08-20	Delivered
	104	Sneha	Reddy	1005	2024-08-12	Cancelled
	105	Vikram	Singh	1006	2024-08-15	Delivered
	106	Ananya	Iyer	1007	2024-08-18	Pending
	107	Karan	Mehta	1009	2024-08-25	Shipped
	108	Divya	Nair	1010	2024-08-28	Delivered

Q21. Write a query using JOINS across three tables (orders → order_items → products) to show: order_id, product_name, quantity, unit_price, and discount_pct for each order item.

```

SELECT
    o.order_id,
    p.product_name,
    oi.quantity,
    oi.unit_price,
    oi.discount_pct
FROM orders o
JOIN order_items oi
ON o.order_id = oi.order_id
JOIN products p
ON oi.product_id = p.product_id;

```

Result Grid					
		Filter Rows:			Export:
				Wrap Cell Content:	
	order_id	product_name	quantity	unit_price	discount_pct
▶	1001	Wireless Earbuds	2	1499.00	0.00
	1006	Wireless Earbuds	1	1499.00	10.00
	1002	Cotton T-Shirt	1	799.00	0.00
	1003	Smart Watch	1	2999.00	0.00
	1005	Smart Watch	1	2999.00	0.00
	1003	Running Shoes	1	4599.00	5.00
	1006	Running Shoes	1	4599.00	5.00
	1004	Bluetooth Speaker	1	3499.00	0.00
	1009	Bluetooth Speaker	1	3499.00	0.00
	1007	Bedsheet Set	1	1299.00	0.00
	1010	Bedsheet Set	1	1299.00	0.00
	1001	Laptop Stand	1	899.00	10.00
	1008	Laptop Stand	1	899.00	0.00
	1009	Cushion Covers (...)	2	599.00	15.00
	1010	Cushion Covers (...)	1	599.00	0.00

Q22. Explain the difference between LEFT JOIN and RIGHT JOIN with an example from this schema. When would you use a FULL OUTER JOIN?

LEFT JOIN

Returns:

- All rows from the left table.
- Matching rows from the right table.
- Non-matching rows on the right become NULL.

Example:

```
SELECT *
FROM customers c
LEFT JOIN orders o
ON c.customer_id = o.customer_id;
```

Result:

- All customers appear.
- Customers without orders still appear with NULL values.

Result Grid													
Filter Rows:													
Export: Wrap Cell Content:													
customer_id	first_name	last_name	email	city	state	join_date	is_premium	order_id	customer_id	order_date	status	total_amount	
101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1	1001	101	2024-08-01	Delivered	4498.00	
101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1	1004	101	2024-08-10	Delivered	3499.00	
102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0	1002	102	2024-08-03	Delivered	799.00	
103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1	1003	103	2024-08-05	Shipped	7498.00	
103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1	1008	103	2024-08-20	Delivered	899.00	
104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0	1005	104	2024-08-12	Cancelled	2999.00	
105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1	1006	105	2024-08-15	Delivered	5898.00	
106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0	1007	106	2024-08-18	Pending	1299.00	
107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1	1009	107	2024-08-25	Shipped	6098.00	
108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	0	1010	108	2024-08-28	Delivered	1598.00	

RIGHT JOIN

Returns:

- All rows from the right table.
- Matching rows from the left table.
- Non-matching rows on the left become NULL.

Example:

```
SELECT *
FROM customers c
RIGHT JOIN orders o
ON c.customer_id = o.customer_id;
```

Result:

- All orders appear.
- If an order had no matching customer, customer columns would be NULL.

Result Grid													
Filter Rows:													
Export: Wrap Cell Content:													
customer_id	first_name	last_name	email	city	state	join_date	is_premium	order_id	customer_id	order_date	status	total_amount	
101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1	1001	101	2024-08-01	Delivered	4498.00	
102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0	1002	102	2024-08-03	Delivered	799.00	
103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1	1003	103	2024-08-05	Shipped	7498.00	
101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1	1004	101	2024-08-10	Delivered	3499.00	
104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0	1005	104	2024-08-12	Cancelled	2999.00	
105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1	1006	105	2024-08-15	Delivered	5898.00	
106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0	1007	106	2024-08-18	Pending	1299.00	
103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1	1008	103	2024-08-20	Delivered	899.00	
107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1	1009	107	2024-08-25	Shipped	6098.00	
108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	0	1010	108	2024-08-28	Delivered	1598.00	

FULL OUTER JOIN

Returns:

- All rows from both tables.
- Matching rows are combined.
- Unmatched rows from either side appear with NULL values.

Use Case:

- Finding missing or unmatched records.
- Data auditing and reconciliation.

Example:

Customers without Orders + Orders without Customers

MySQL does not directly support FULL OUTER JOIN, but it can be simulated using:

LEFT JOIN

UNION

RIGHT JOIN

Q23. Identify all Foreign Key relationships in the schema. Explain what would happen if you tried to insert an order with customer_id = 999 (which doesn't exist in customers)

Relationship 1

orders.customer_id

REFERENCES customers(customer_id)

Meaning:

- Every order must belong to an existing customer.

Relationship 2

order_items.order_id

REFERENCES orders(order_id)

Meaning:

- Every order item must belong to an existing order.

Relationship 3

order_items.product_id

REFERENCES products(product_id)

Meaning:

- Every order item must reference an existing product.

What happens if you insert an order with customer_id = 999?

Query

```
INSERT INTO orders VALUES (1011, 999, '2024-09-01', 'Pending', 1000.00 );
```

Result

MySQL Error:

Cannot add or update a child row:
a foreign key constraint fails

Explanation

Customer 999 does not exist in the customers table.

The foreign key:

FOREIGN KEY (customer_id)
REFERENCES customers(customer_id)

enforces referential integrity and prevents invalid customer references from being inserted into the orders table.

SECTION E- Advanced Concepts (CASE, ACID, Transactions)

Q24. Write a query using CASE to classify products into price tiers:

- 'Budget' → unit_price < 1000
- 'Mid-Range' → unit_price BETWEEN 1000 AND 3000
- 'Premium' → unit_price > 3000

Display: product_name, unit_price, price_tier.

Query

```
SELECT
    product_name,
    unit_price,
    CASE
        WHEN unit_price < 1000 THEN 'Budget'
        WHEN unit_price BETWEEN 1000 AND 3000 THEN 'Mid-Range'
        ELSE 'Premium'
    END AS price_tier
FROM products;
```


Result Grid			
Filter Rows:			
Export:  Wrap Cell Content: 			
	product_name	unit_price	price_tier
▶	Wireless Earbuds	1499.00	Mid-Range
	Cotton T-Shirt	799.00	Budget
	Smart Watch	2999.00	Mid-Range
	Running Shoes	4599.00	Premium
	Bluetooth Speaker	3499.00	Premium
	Bedsheet Set	1299.00	Mid-Range
	Laptop Stand	899.00	Budget
	Cushion Covers (Set)	599.00	Budget

Q25. Using a CASE statement inside an aggregate function, count how many orders are 'Delivered' vs 'Not Delivered' (all other statuses). Display the result in a single row.

SELECT

SUM(

CASE

WHEN status = 'Delivered' THEN 1

ELSE 0

END

) AS Delivered_Orders,

SUM(

CASE

WHEN status <> 'Delivered' THEN 1

ELSE 0

END

) AS Not_Delivered_Orders

FROM orders;

Result Grid		
Filter Rows:		
Export:  Wrap Cell Content: 		
	Delivered_Orders	Not_Delivered_Orders
▶	6	4

Q26. Explain each letter of ACID:

- **A – Atomicity**
- **C – Consistency**
- **I – Isolation**
- **D – Durability**

Give a real-world example (e.g., bank transfer) showing why each property is important.

ACID properties ensure that database transactions are reliable and consistent.

A — Atomicity

Definition:

A transaction is treated as a single unit. Either all operations succeed or none of them are applied.

Bank Transfer Example:

Suppose ₹1000 is transferred from Account A to Account B.

Steps:

1. Deduct ₹1000 from A
2. Add ₹1000 to B

If the system crashes after Step 1 but before Step 2:

Without Atomicity:

- A loses money
- B doesn't receive money

With Atomicity:

- Entire transaction is rolled back.

C — Consistency

Definition:

The database must always remain in a valid state before and after a transaction.

Example:

Before transfer:

A = ₹5000

B = ₹3000

Total = ₹8000

After transfer:

A = ₹4000

B = ₹4000

Total = ₹8000

The database rules remain valid.

I — Isolation

Definition:

Multiple transactions should not interfere with each other.

Example:

Two ATM withdrawals happen simultaneously.

Without Isolation:

- Both transactions may read the same balance.
- Incorrect balance may result.

With Isolation:

- Each transaction behaves as if it is running alone.

D — Durability

Definition:

Once a transaction is committed, the changes are permanently saved.

Example:

After a successful bank transfer:

COMMIT;

Even if:

- Power fails
- Database restarts
- Server crashes

the committed data remains stored.

Q27. Write a SQL transaction that does the following atomically:

- 1. Insert a new order (order_id=1011, customer_id=102, today's date, 'Pending', 1598.00)**
- 2. Insert two order items for that order**

3. Update the stock_qty of the purchased products

4. If any step fails, ROLLBACK the entire transaction. Otherwise, COMMIT.

Write the complete BEGIN...COMMIT/ROLLBACK block.

Transaction

START TRANSACTION;

Step 1: Insert New Order

```
INSERT INTO orders (order_id, customer_id, order_date, status, total_amount) VALUES(1011, 102, CURDATE(), 'Pending', 1598.00);
```

Step 2: Insert Order Item 1

```
INSERT INTO order_items(item_id, order_id, product_id, quantity, unit_price, discount_pct) VALUES (5016, 1011, 202, 1, 799.00, 0);
```

Step 3: Insert Order Item 2

```
INSERT INTO order_items(item_id, order_id, product_id, quantity, unit_price, discount_pct) VALUES(5017, 1011, 208, 1, 599.00, 0);
```

Step 4: Update Product Stock

```
UPDATE products SET stock_qty = stock_qty - 1 WHERE product_id = 202;
```

```
UPDATE products SET stock_qty = stock_qty - 1 WHERE product_id = 208;
```

-- If all statements execute successfully

```
COMMIT;
```

-- If any statement fails

```
ROLLBACK;
```

CONCLUSION:

This project demonstrated the use of SQL for data retrieval, filtering, aggregation, joins, constraints, indexing, and transaction management in an e-commerce database. Through the ShopEase dataset, business insights such as customer behavior, product performance, and order trends were analyzed. The project also highlighted the importance of database constraints, indexing strategies, ACID properties, and transaction control for maintaining data integrity and improving query performance.